

Secure Speculative Decoding for Large Language Models

Yichi Zhang¹, Zhiqi Wang¹, Neil Zhenqiang Gong², Yuchen Yang¹

¹The Pennsylvania State University, ²Duke University

{yichi.zhang, zhiqi.wang, yuchen.yang}@psu.edu, neil.gong@duke.edu

Abstract—Speculative decoding accelerates inference for a large language model (LLM), referred to as the *target model*, by first using a smaller model, referred to as the *draft model*, to generate candidate tokens and then verifying them with the target model for acceptance or rejection. Prior studies primarily focused on the efficiency-utility trade-off of speculative decoding, e.g., lossy speculative decoding, leaving its security implications largely unexplored.

In this work, we bridge this gap by providing the *first* systematic study of the security implications of speculative decoding. Through a large-scale measurement study, we reveal a pronounced security-utility asymmetry: across a wide range of lossy speculative decoding methods, improvements in inference efficiency come at a disproportionately high cost to security, with attack success rates for jailbreak and prompt injection attacks increasing much faster than utility degrades.

We then propose SECURESD, a new theory-guided speculative decoding method that enhances security while maintaining efficiency and utility. Specifically, our theoretical analysis reveals that security degradation primarily originates from the early tokens generated by the draft model. Motivated by this insight, SECURESD applies a stricter verification criterion to draft-model tokens at early decoding positions. Extensive experiments on both security and utility benchmarks demonstrate that SECURESD significantly improves security while preserving efficiency and utility compared to existing speculative decoding methods.¹

1. Introduction

As LLMs are increasingly deployed in interactive applications, autoregressive decoding has become a major latency bottleneck because each output token typically requires a separate forward pass through a large model. As shown in Algorithm 1, speculative decoding addresses this bottleneck by using a smaller *draft model* to quickly generate candidate tokens for a larger LLM, referred to as the *target model*; the target model then verifies these candidates in parallel and either accepts them or rejects them before continuing generation [1], [2]. By accepting multiple tokens in a single decoding step, speculative decoding can substantially reduce decoding latency. This technique has moved beyond research

prototypes: Google reports applying speculative decoding in multiple products, including AI Overviews in Google Search [3] and Gemma 4 models [4]. Production-oriented serving stacks such as vLLM [5], NVIDIA Triton/TensorRT-LLM [6], Hugging Face Inference [7], and SGLang [8] also provide built-in speculative decoding support.

Existing work on speculative decoding has mainly focused on improving inference efficiency while preserving utility. Recent methods increase speedup by improving draft models [9], [10], [11], [12], [13], using richer proposal structures [14], [15], [16], [17], or adopting lossy verification strategies [18], [19], [20], [21] that accept more draft tokens per decoding step. Some methods further relax the verification criterion [22], [23], [24], [25], [26], explicitly trading exact target-model behavior for higher throughput. However, these methods are still primarily evaluated through the efficiency-utility trade-off, using metrics such as speedup, average acceptance length, perplexity, task accuracy, or human preference. *To the best of our knowledge, the safety and security implications of efficiency-oriented speculative decoding under adversarial inputs have not been systematically studied.* This gap matters because relaxed verification may accept draft tokens that the target model would otherwise reject. In safety settings, e.g., jailbreak [27], [28], [29], [30], and security settings, e.g., prompt injection [31], such early deviations can steer the response from refusal toward compliance, causing safety and security risks that remain invisible to utility benchmarks. For simplicity, we use *security* to refer to both throughout the introduction.

In this paper, we bridge this gap by providing the *first* systematic study of the security implications of lossy speculative decoding. Across a range of speculative decoding methods, making token acceptance more aggressive improves inference efficiency with only modest degradation on standard utility benchmarks, but causes a disproportionately large increase in attack success rates on security benchmarks. In particular, jailbreak and prompt injection attacks become much more successful even when task accuracy, output quality, or other utility metrics remain relatively stable. This shows that security degradation is not simply another form of utility degradation: lossy speculative decoding can introduce small changes in the generation trajectory that are largely tolerated by general-purpose benchmarks but have much larger effects on security-critical behavior.

To understand this asymmetry, we analyze how accepted draft tokens affect autoregressive generation. Our analysis

¹ The code and benchmarks for SECURESD are publicly available at <https://github.com/YichiCS/Secure-Speculative-Decoding>.

shows that security degradation depends on two dimensions: (i) how far the accepted tokens deviate from the target model’s original distribution, and (ii) at which token position these deviations occur in the output sequence. In particular, draft tokens accepted at early decoding positions have a disproportionate effect because they set the initial direction of the response. For adversarial prompts, an early deviation can shift generation from refusal toward compliance. This positional sensitivity explains why security can degrade much faster than utility.

Motivated by this insight, we propose SECURESD, a theory-guided and position-aware speculative decoding method that applies stricter verification to early draft tokens while retaining efficient verification later in the generation process. By correcting only the positions most responsible for security degradation, SECURESD avoids unnecessary verification overhead across the full response. Experiments on five utility and security benchmarks show that SECURESD reduces attack success rates of jailbreak and prompt injection attacks by up to 92.4%, while retaining 99.8% of the decoding speedup and keeping utility within 98.4% of existing speculative decoding methods. Ablation studies further confirm that focusing verification on early positions is key to improving the security–utility–efficiency trade-off.

Contribution. We make the following contributions:

- We provide the *first* systematic study of the security implications of lossy speculative decoding, revealing a strong **security–utility asymmetry**: attack success rates rise much faster than utility degrades.
- We explain this asymmetry through theoretical and empirical analysis, showing that early draft tokens play an important role in security degradation.
- We propose SECURESD, a theory-guided, position-aware speculative decoding method that improves security while preserving efficiency and utility.

2. Related Work

2.1. Speculative Decoding

Speculative decoding [1], [2] accelerates LLM generation by using a small draft model to propose candidate tokens and a larger target model to verify them in parallel, reducing expensive target-model computation while strictly preserving the target model’s behavior. As shown in Algorithm 1, each decoding round proceeds in two phases: the draft model first autoregressively proposes k candidate tokens, and the target model then computes their probabilities in a single parallel forward pass. Each candidate $\tilde{x}_i$ is accepted with probability $\min\{1, q_i(\tilde{x}_i)/p_i(\tilde{x}_i)\}$; upon the first rejection, a replacement token is sampled from the normalized residual distribution $(q_i - p_i)_+$ and the round terminates. If all k candidates pass, one extra token is sampled from q_{k+1} , so each round commits between 1 and $k + 1$ tokens whose distribution provably matches sampling from the target model directly.

Algorithm 1: Vanilla Speculative Decoding

Input: Draft model p , target model q , prefix $x_{1:t}$, speculation length k

Output: Tokens sampled from speculative decoding

```

1 for  $i = 1$  to  $k$  do
2    $p_i \leftarrow p(\cdot \mid x_{1:t}, \tilde{x}_{1:i-1})$ ;
3    $\tilde{x}_i \sim p_i$ ;
4  $\{q_i\}_{i=1}^{k+1} \leftarrow \text{TARGETFORWARD}(x_{1:t}, \tilde{x}_{1:k})$ ;
5 for  $i = 1$  to  $k$  do
6    $r \sim \mathcal{U}(0, 1)$ ;
7   if  $r \leq \min\{1, q_i(\tilde{x}_i)/p_i(\tilde{x}_i)\}$  then
8      $x_{t+i} \leftarrow \tilde{x}_i$ ;
9   else
10     $x_{t+i} \sim \frac{(q_i - p_i)_+}{\sum_x (q_i(x) - p_i(x))_+}$ ;
11  return  $x_{t+1}, \dots, x_{t+i}$ ;
12  $x_{t+k+1} \sim q_{k+1}$ ;
13 return  $x_{t+1}, \dots, x_{t+k+1}$ ;
```

Recent works [22], [23], [24], [25], [26] further improve efficiency by relaxing the verification step. Instead of strictly preserving the target distribution, these methods accept more draft tokens when they appear sufficiently close to what the target model would have produced. This increases the acceptance rate and decoding speed with a small distributional shift from the original target model.

Specifically, LossySD [1] relaxes the verification rule, allowing more draft tokens to pass verification under a controlled approximation. BiLD [22] uses model confidence to decide when to continue relying on the draft model and when to roll back to the target model. SpecCascade [23] formulates verification as a cascade decision, determining when the system can trust the draft model and when it should defer to the target model. FSD [24] is designed to accept a draft token when the draft and target distributions are sufficiently similar. FLy [25] uses uncertainty in the target distribution as a signal for relaxation, permitting more relaxation when the target model is confident enough. MARS [26] relies on the structure of the target model’s logits to determine whether a draft token is consistent with the target model’s predictions.

However, these works mainly focus on optimizing the efficiency–utility trade-off, leaving unclear how the distributional shifts introduced by relaxed verification affect security. We bridge this research gap in this paper.

2.2. Attacks on LLMs

Jailbreak Attacks. Jailbreak attacks are designed to guide LLMs to circumvent their safety rules and respond to queries that would otherwise be refused. Malicious users can craft natural-language prompts that reframe harmful requests through roleplay, fictional scenarios, or privilege escalation, such as “Do-Anything-Now” prompts [32], “developer mode” prompts, and character-roleplay prompts [33].

More recent work formalizes these methods through persuasion [34], highlighting that LLMs are vulnerable to natural language prompting techniques.

Another line of jailbreak work relies on adaptive prompt-crafting algorithms [27], [35], [36], [37] that use feedback from the target model. AutoDAN [35] optimizes human-crafted jailbreak prompts with a genetic algorithm by iteratively querying the target LLM and improving the prompt. Greedy Coordinate Gradient (GCG) [27] optimizes an adversarial suffix to increase the likelihood of a compliant response from the target model, making it an iterative attack.

Prompt Injection Attacks. Prompt injection attacks target the instruction-following behavior of LLM-integrated applications. In these attacks, an adversary supplies additional instructions that conflict with the intended task or higher-level system instructions [31], [38], [39], [40]. Earlier work studies direct prompt injection [39], where a user attempts to override the high-level instructions of an LLM-based application with commands such as “ignore previous instructions.” Later work shows that malicious instructions can also be planted in external sources, such as webpages, documents, or tool outputs, and then hijack a session between the benign user and LLM-based agents. This setting is known as indirect prompt injection [31], [38], [40]. Liu et al. [31] formalize in-the-wild prompt injection techniques into a unified framework and show that combining existing techniques improves attack success rates. Adaptive or automatic jailbreak algorithms [27], [37] can also be used to generate successful prompt injection prompts.

In this paper, we use jailbreak and prompt injection benchmarks to quantify the underexplored security-utility asymmetry introduced by relaxed speculative decoding: utility can remain largely stable while security degrades rapidly relative to the target model’s security performance.

3. Threat Model

System Model. We consider a realistic deployment scenario in which the LLM infrastructure integrates speculative decoding with relaxed verification for further speedup. The system uses a large target model to generate responses and a cheaper draft model to propose candidate tokens. For the same input X , we compare the output of an LLM system with relaxed speculative decoding against that of an otherwise identical system with vanilla (i.e., lossless) speculative decoding. Our goal is to measure how relaxed verification changes the system’s behavior relative to the vanilla baseline, and how much target-model performance it retains, treating the target model as the upper bound.

We focus on lossy speculative decoding systems that use relaxed verification to trade exact target-model behavior for throughput. Mainstream serving frameworks, including vLLM, NVIDIA TensorRT-LLM, and SGLang, support this setting [5], [6], [8]. We quantify relaxation by acceptance rate (AR) and evaluate its practical regime in Section 4.

Attacker’s Objectives. We use adversarial objectives to define the safety and security stress-test setting. For a given

query X , it produces an incorrect or unintended response Y . We primarily consider two attack scenarios.

- **Jailbreak Attacks.** The attacker carefully crafts a query X to induce the LLM system to generate responses Y that violate human values or established safety policies.
- **Prompt Injection Attacks.** The attacker carefully crafts a query X to manipulate the LLM system such that it produces an unintended response Y or causes a downstream application to execute potentially harmful actions Y .

Attacker’s Capabilities. We assume a black-box threat model in which the attacker can interact with the LLM system only through its public API. Specifically, the attacker can issue queries to an LLM system that employs relaxed speculative decoding and observe the generated responses.

Attacker’s Knowledge. We assume that the attacker possesses partial knowledge of the target model, such as the target LLM’s family or type (e.g., GPT-5.5). However, we do not assume knowledge of the exact relaxed speculative decoding configuration, including the draft model, verification rule, relaxation threshold, or decoding randomness. Since the same inputs are evaluated under the target model with and without relaxed speculative decoding, the measured security gap reflects the effect of relaxed verification rather than an exploitation of internal implementation details.

4. Measurement: Security–Utility Asymmetry

We begin with a measurement study of how speculative decoding affects security performance as the token acceptance strategy becomes more aggressive.

4.1. Preliminaries on Speculative Decoding

Speculative decoding methods share the same high-level draft-then-verify paradigm, but they differ in how the verifier decides whether to accept a proposed draft token and how generation proceeds after rejection. To compare these methods under a common framework, we first define a unified verifier abstraction.

Definition 1 (Speculative Decoding Verifier). *A speculative decoding verifier can be represented as $S(q, p, \alpha, R)$, where the system operates over a sequence of draft positions and sequentially decides whether to accept or reject each draft token. At position t , conditioned on the current prefix $x_{<t}$, the target model and the draft model define the following conditional distributions:*

$$q_t(x) = q(x_t = x \mid x_{<t}), \quad (1)$$

$$p_t(x) = p(x_t = x \mid x_{<t}). \quad (2)$$

Here, q_t specifies the target distribution that the decoding procedure aims to preserve, while p_t specifies the draft distribution for efficient token generation.

Given a draft token x at position t , the system applies an acceptance function

$$\alpha_t(x) : \mathcal{V} \rightarrow [0, 1], \quad (3)$$

TABLE 1: **Average performance retention rate (PRR) and speedup (SPD) of lossless and six lossy speculative decoding methods across security and utility benchmarks.** For each speculative decoding method, we sweep its method-specific hyperparameters θ to obtain different ARs. We then report the average PRR and SPD as a function of AR. We also report the PRR and SPD of lossless speculative decoding for comparison with the average performance of lossy speculative decoding.

Relaxed Verifier	Model Family	Safety/Security Benchmarks				Utility Benchmarks			
		Jailbreak-SD		OpenPromptInjection		HumanEval		GSM8K	
		PRR $\uparrow$	SPD $\uparrow$	PRR $\uparrow$	SPD $\uparrow$	PRR $\uparrow$	SPD $\uparrow$	PRR $\uparrow$	SPD $\uparrow$
LossySD [1]	Qwen3-0.6B/8B	.708	1.68	.657	1.56	.834	1.89	.925	1.91
	Llama3-1B/8B	.693	1.84	.763	1.70	.803	1.96	.878	2.03
BiLD [22]	Qwen3-0.6B/8B	.674	1.50	.664	1.35	.856	1.63	.945	1.74
	Llama3-1B/8B	.568	1.59	.718	1.46	.748	1.73	.903	1.71
SpecCascade [23]	Qwen3-0.6B/8B	.566	1.60	.553	1.45	.771	1.79	.831	1.83
	Llama3-1B/8B	.651	1.76	.720	1.58	.781	1.87	.855	1.90
FSD [24]	Qwen3-0.6B/8B	.927	1.60	.759	1.56	.865	1.58	.940	1.59
	Llama3-1B/8B	.610	1.85	.734	1.76	.850	1.64	.945	1.50
FLy [25]	Qwen3-0.6B/8B	.826	1.71	.694	1.58	.935	1.91	.951	1.96
	Llama3-1B/8B	.752	1.90	.752	1.79	.839	1.98	.915	2.06
MARS [26]	Qwen3-0.6B/8B	.993	1.13	.872	1.13	.985	1.78	.998	1.86
	Llama3-1B/8B	.817	1.48	.845	1.46	.863	1.91	.996	1.88
Average Lossy SD	Qwen3-0.6B/8B	.782	1.54	.700	1.44	.874	1.76	.932	1.82
	Llama3-1B/8B	.682	1.74	.755	1.63	.814	1.85	.915	1.85
Lossless SD	Qwen3-0.6B/8B	1.019	1.05	.915	1.06	.947	1.65	1.000	1.81
	Llama3-1B/8B	.909	1.33	.983	1.36	.933	1.85	.976	1.76

which determines the probability with which the proposed token is accepted under the current prefix. If the proposed token is rejected, the system samples a replacement token from a recovery distribution

$$R_t(x) = R(x_t = x \mid x_{<t}). \quad (4)$$

Here, R_t specifies the distribution used to recover from rejection and continue generation.

This verifier definition can be broadly applied to current frameworks such as LossySD [1], BiLD [22], SpecCascade [23], FSD [24], FLy [25], and MARS [26]. We show how a speculative decoding method can be characterized by these four parameters in Table 2. Tuning the hyperparameters of this method allows us to modulate the aggressiveness of the corresponding token-acceptance strategies.

4.2. Experimental Setup

Benchmarks. We evaluate the system on utility, safety, and security benchmarks. For utility evaluation, we use two benchmarks commonly adopted in speculative decoding studies: 164 Python programming problems from HumanEval [41] to assess code generation, and 200 problems sampled from GSM8K [42] to assess mathematical reasoning. We consider jailbreak for safety settings and prompt injection for security settings. For jailbreak evaluation, we construct a benchmark named Jailbreak-SD of 200 examples from four widely used datasets: WildJailbreak [28], JailbreakBench [29], HarmBench [30], and AdvBench [27]. For prompt injection, we use OpenPromptInjection [31] to evaluate general prompt injection attacks. We sample two

TABLE 2: **Instantiation of relaxed verifiers.** We instantiate six speculative decoding methods with relaxed verification under Definition 1.

Method	q_t	p_t	$\alpha_t(x)$	R_t
LossySD [1]	q_t	p_t	$\min\{1, q_t(x)/p_t(x) + \epsilon\}$	$\rho_t(q_t; x)^1$
BiLD [22]	q_t	p_t	$\mathbf{1}[\max_v p_t(v) \geq \text{fb}] \cdot \mathbf{1}[q_t(x) \geq e^{-rb} \vee x = \hat{x}_t]$	$\delta_{\hat{x}_t}$
SpecCascade [23]	π_t^r	p_t	$\min\{1, \pi_t^r(x)/p_t(x)\}$	$\rho_t(\pi_t^r; x)$
FSD [24]	q_t	p_t	$\mathbf{1}[D_r(q_t \ p_t) \leq \tau]$	q_t
FLy [25]	q_t	p_t	$\mathbf{1}[x = \hat{x}_t] \vee \mathbf{1}[\tilde{H}(q_t) \geq \theta \wedge \neg \text{win}_t^W]$	q_t
MARS [26]	q_t	p_t	$\mathbf{1}[x = \hat{x}_t] \vee \mathbf{1}[x = \text{top-2}(q_t) \wedge z_2/z_1 > \tau]$	$\delta_{\hat{x}_t}$

¹ $\rho_t(\mu; x) = (\mu(x) - p_t(x))_+ / \sum_v (\mu(v) - p_t(v))_+$ denotes the positive-residual recovery distribution, where $(f)_+ = \max\{f, 0\}$.

instructions from different sources across seven task datasets and combine them using five attack strategies, resulting in a benchmark of 200 samples, with 40 samples for each attack strategy. We also conduct a full evaluation on Agent-Dojo [43] to assess more realistic prompt injection scenarios in long-context tasks.

Metrics. We evaluate speculative decoding using three metrics that capture its efficiency, utility, and security.

- **Acceptance Rate (AR).** AR measures acceptance aggressiveness as the fraction of draft tokens accepted. We sweep the method-specific hyperparameters in Table 2 to obtain AR values from conservative verification to aggressive acceptance, forming an approximately continuous scale between 0 and 1.

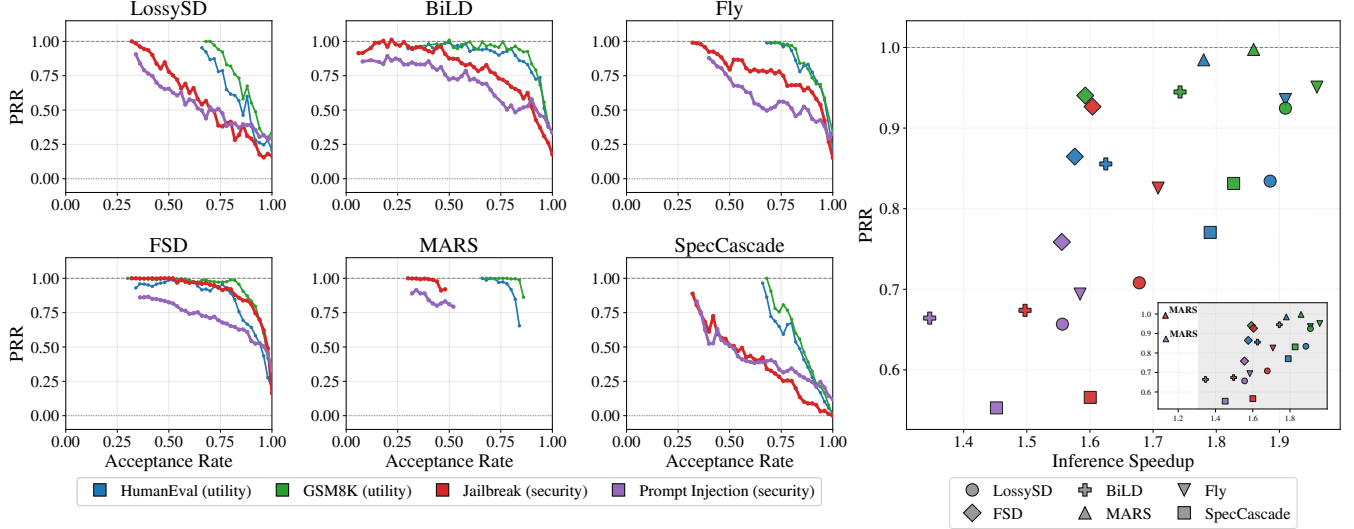


Figure 1: **Security-Utility Asymmetry under Speculative Decoding.** The left panel shows the PRR-AR curves of six speculative decoding methods on four benchmarks. The right panel shows the trade-off between PRR and SPD. The separation between the curves and scatter points for security and utility tasks reveals the security-utility asymmetry.

- **Performance Retention Rate (PRR).** To compare target-model performance retention across tasks with different score scales, we orient all *raw scores* so that **higher is better**: Pass@1 for HumanEval, Accuracy for GSM8K, attack failure rate (AFR) for Jailbreak-SD and OpenPrompt-Injection, and utility under attack (UUA) for AgentDojo. We use the official evaluators to compute Pass@1, Accuracy, and UUA. For AFR, we follow the StrongREJECT [44] setting and use GPT-4o-mini as an LLM-as-a-judge [45] to determine whether the attack succeeds. Let x be the raw score of a certain speculative decoding method, and let x_{target} and x_{draft} be the raw scores of the target and draft models under standard decoding. We define

$$\text{PRR} = \frac{x - x_{\text{draft}}}{x_{\text{target}} - x_{\text{draft}}}, \quad (5)$$

which measures the fraction of the target performance retained by speculative decoding: PRR = 0 matches the draft model, and PRR = 1 matches the target model. We also report raw scores for direct interpretation in Table 3.

- **Inference Speedup (SPD).** SPD is the ratio between the wall-clock decoding time of the target model under standard decoding and that of speculative decoding. Larger values indicate faster inference, e.g., SPD = 2 means that speculative decoding achieves a 2× speedup against standard autoregressive decoding of the target model.

Speculative Decoding Methods. We consider the lossless method and six lossy speculative decoding methods: LossySD [1], BiLD [22], FLY [25], FSD [24], MARS [26], and SpecCascade [23]. We use a default speculation length of $k = 5$, meaning that the draft model proposes five candidate tokens at each decoding step. Unless otherwise specified, we use a generation batch size of 32 and set the decoding temperature to 0, corresponding to greedy

TABLE 3: **Raw Model Performance.** We report the task performance of four models across five benchmarks, highlighting the performance gap between the draft and target models. We use these results to compute PRR.

Task (Metric)	Qwen3		Llama3	
	0.6B	8B	1B	8B
Jailbreak-SD (AFR↑)	.195	.895	.105	.915
OpenPromptInjection (AFR↑)	.030	.820	.060	.830
AgentDojo (UUA↑)	.078	.264	—	—
HumanEval (Pass@1↑)	.396	.854	.427	.701
GSM8K (Accuracy↑)	.500	.930	.410	.820

decoding. These are widely adopted default settings. For notational simplicity, we use θ to abstractly denote the tunable hyperparameters of each speculative decoding method.

Models. We evaluate four draft-target model pairs. We pair Qwen3-0.6B (i.e., draft model) with Qwen3-8B and Qwen3-32B as target models, and Llama-3.2-1B-Instruct (i.e., draft model) with Llama-3.1-8B-Instruct and Llama-3.3-70B-Instruct as target models. We report the standard evaluation metrics of these models on the four benchmarks in Table 3, and use these results to compute PRR in the subsequent sections.

Environment. All experiments are conducted on a server equipped with an AMD EPYC 9334 32-Core Processor running Ubuntu 22.04.5 LTS, with four NVIDIA RTX PRO 6000 Blackwell Workstation Edition GPUs.

4.3. Security Degrades Faster than Utility

Lower Security Performance Retention. We first investigate how lossy speculative decoding affects security and utility benchmarks across a broad range of verifier settings.

Table 1 reports the PRR and SPD of different lossy speculative decoding methods, averaged over hyperparameter sweeps that span diverse ARs. We have two observations:

- *Lossy speculative decoding retains lower PRR on security benchmarks than on utility benchmarks.* For Qwen3-0.6B/8B, the average PRR is 0.744 on the two security benchmarks, Jailbreak and Prompt Injection, compared with 0.905 on the two utility benchmarks, HumanEval and GSM8K. The same trend holds for Llama3-1B/8B, with average PRRs of 0.719 on security benchmarks and 0.866 on utility benchmarks.
- *The lower PRR on security benchmarks does not come with a higher SPD.* For Qwen3-0.6B/8B, lossy speculative decoding achieves only $1.49\times$ average SPD on security benchmarks, compared with $1.79\times$ on utility benchmarks. For Llama3-1B/8B, the average SPD is $1.69\times$ on security benchmarks and $1.85\times$ on utility benchmarks. Therefore, security benchmarks suffer larger performance degradation, but do not receive higher SPD in return.

Earlier Security Degradation. We next measure how fast security and utility performance degrade as the verifier accepts more draft tokens, i.e., with a higher AR. Figure 1 shows how the PRR varies with the AR on utility and security benchmarks. Moving right along a curve corresponds to a more aggressive acceptance strategy with higher SPD, but the generated sequence may deviate further from the target model’s original behavior. We have two observations:

- *Security PRR is lower at comparable ARs.* PRR decreases as the AR increases on both utility and security benchmarks, indicating that a more aggressive relaxed verifier introduces greater behavioral deviation from the target model. However, this degradation is consistently larger for security tasks. Under the same verifier and at comparable ARs, security benchmarks retain substantially less target model performance than utility benchmarks.
- *Security PRR drops much earlier.* Security degradation also begins at lower ARs. For several methods, utility PRR on GSM8K and HumanEval remains close to the baseline until the verifier becomes highly aggressive, e.g., at ARs around 0.7. In contrast, security PRR often starts to decline at much lower ARs, e.g., around 0.3.

These observations show that speculative decoding with relaxed verifiers can weaken jailbreak or prompt injection defenses even when performance on general utility benchmarks shows only limited degradation.

Practical Operating Regime. Compared with lossless SD, lossy SD raises average SPD from $1.05\times$ to $1.49\times$ on Qwen and from $1.35\times$ to $1.69\times$ on Llama. Combined with the thresholds above, $\text{AR} \in [0.3, 0.7]$ identifies the regime where additional speedup is meaningful and safety/security degradation has emerged.

Takeaway 1. Lossy speculative decoding exhibits a security–utility asymmetry: at comparable ARs, it can preserve utility while incurring earlier and sharper degradation on security. These results suggest that efficiency–utility evaluation alone is insufficient for assessing speculative

Algorithm 2: SECURESD

Input: Relaxed verifier S with acceptance functions $\{\alpha_i\}_{i=1}^k$ and recovery distributions $\{R_i\}_{i=1}^{k+1}$, prefix $x_{1:t}$, speculation length k , verification correction schedule $\{\eta_j\}_{j \geq 1}$

Output: Tokens sampled under SECURESD

```

1 for  $i = 1$  to  $k$  do
2    $p_i \leftarrow p(\cdot \mid x_{1:t}, \tilde{x}_{1:i-1})$ ;
3    $\tilde{x}_i \sim p_i$ ;
4  $\{q_i\}_{i=1}^{k+1} \leftarrow \text{TARGETFORWARD}(x_{1:t}, \tilde{x}_{1:k})$ ;
5 for  $i = 1$  to  $k$  do
6    $\alpha_i^{\text{sd}} \leftarrow \min\{\mathbf{1}, q_i/p_i\}$ ;
7    $R_i^{\text{sd}} \leftarrow \frac{(q_i - p_i)_+}{\sum_x (q_i(x) - p_i(x))_+}$ ;
8    $\alpha_i^\eta \leftarrow (1 - \eta_{t+i})\alpha_i^{\text{sd}} + \eta_{t+i}\alpha_i$ ;
9    $r \sim \mathcal{U}(0, 1)$ ;
10  if  $r \leq \alpha_i^\eta(\tilde{x}_i)$  then
11     $x_{t+i} \leftarrow \tilde{x}_i$ ;
12  else
13     $R_i^\eta \leftarrow (1 - \eta_{t+i})R_i^{\text{sd}} + \eta_{t+i}R_i$ ;
14     $x_{t+i} \sim R_i^\eta$ ;
15  return  $x_{t+1}, \dots, x_{t+k}$ ;
16  $R_{k+1}^\eta \leftarrow (1 - \eta_{t+k+1})q_{k+1} + \eta_{t+k+1}R_{k+1}$ ;
17  $x_{t+k+1} \sim R_{k+1}^\eta$ ;
18 return  $x_{t+1}, \dots, x_{t+k+1}$ ;

```

decoding in security-sensitive deployments.

5. SECURESD

We now propose SECURESD, a verification correction mechanism that can be seamlessly integrated into any speculative decoding pipeline. SECURESD mitigates the performance degradation introduced by relaxed speculative decoding verifiers, particularly on security-critical tasks.

5.1. SECURESD Overview

Algorithm 2 presents SECURESD, a speculative decoding algorithm that incorporates verification correction. The algorithm is formulated as a single-step procedure, and the complete decoding process can be constructed by invoking it repeatedly. The input consists of a relaxed verifier S , the current prefix $x_{1:t}$, a speculation length k , and a verification correction schedule $\{\eta_j\}_{j \geq 1}$. The verifier S exposes acceptance functions $\{\alpha_i\}_{i=1}^k$ and recovery distributions $\{R_i\}_{i=1}^{k+1}$. The key idea of SECURESD is to use a correction schedule to augment the relaxed verifier with a lossless reference path induced by the standard verifier. We provide both theoretical and empirical analysis of this design in Section 6.

Draft Generation and Scoring (Lines 1–4). Given the prefix $x_{1:t}$, the draft model first generates k speculative

tokens autoregressively. At speculative position i , it forms the draft distribution

$$p_i = p(\cdot \mid x_{1:t}, \tilde{x}_{1:i-1}), \quad (6)$$

and samples $\tilde{x}_i \sim p_i$ (Lines 1–3). These tokens are tentative and are not committed to the output sequence until they pass verification. The target model is then applied in a single forward pass to the concatenation of the current prefix and the speculative draft tokens, yielding target distributions $\{q_i\}_{i=1}^{k+1}$ for the k speculative positions and the subsequent extra-token position (Line 4). This single target forward pass preserves the parallelization benefit of speculative decoding.

Lossless Reference of the Standard Verifier (Lines 6–7).

For each token position, SECURESD constructs the lossless reference of the standard verifier [1]. The acceptance rule is

$$\alpha_i^{\text{sd}} = \min\{\mathbf{1}, q_i/p_i\}, \quad (7)$$

and the corresponding recovery distribution is

$$R_i^{\text{sd}} = \frac{(q_i - p_i)_+}{\sum_x (q_i(x) - p_i(x))_+}. \quad (8)$$

Both of these ensure that speculative decoding remains distributionally lossless.

Verification Correction (Lines 8–18). SECURESD interpolates the standard and relaxed verifiers at each token position. The mixed acceptance probability is defined as

$$\alpha_i^\eta = (1 - \eta_{t+i})\alpha_i^{\text{sd}} + \eta_{t+i}\alpha_i. \quad (9)$$

The proposed draft token $\tilde{x}_i$ is accepted if a uniform random variable $r \sim \mathcal{U}(0, 1)$ is no larger than $\alpha_i^\eta(\tilde{x}_i)$ (Lines 9–11). Upon acceptance, the algorithm sets $x_{t+i} \leftarrow \tilde{x}_i$. If the token is rejected, verification stops immediately. The algorithm then constructs a mixed recovery distribution at the rejected position:

$$R_i^\eta = (1 - \eta_{t+i})R_i^{\text{sd}} + \eta_{t+i}R_i, \quad (10)$$

samples $x_{t+i} \sim R_i^\eta$, and returns the accepted prefix together with this recovery token, as shown in Lines 13–15. This left-to-right stopping rule preserves the causal structure of autoregressive decoding, since all draft tokens after a rejection depend on a token that has not been accepted.

If all k draft tokens are accepted, SECURESD constructs the sampling distribution for the next position by interpolating between the target distribution q_{k+1} and the relaxed verifier’s recovery distribution R_{k+1} :

$$R_{k+1}^\eta = (1 - \eta_{t+k+1})q_{k+1} + \eta_{t+k+1}R_{k+1}. \quad (11)$$

It then samples the extra token $x_{t+k+1} \sim R_{k+1}^\eta$ and returns $x_{t+1}, \dots, x_{t+k+1}$ (Lines 16–18). Therefore, each invocation of SECURESD advances generation by at least one token and by at most $k + 1$ tokens.

5.2. Verification Correction Schedule $\{\eta_j\}_{j \geq 1}$

The verification correction schedule $\{\eta_j\}_{j \geq 1}$ in Algorithm 2 determines how SECURESD interpolates between the standard verifier and the relaxed verifier. The index j denotes the *global* decoding position. When $\eta_j = 0$, the verification procedure reduces to standard speculative decoding. When $\eta_j = 1$, SECURESD recovers the relaxed verifier S and directly applies its acceptance and recovery rules. For $0 < \eta_j < 1$, SECURESD interpolates between the two verification rules, allowing it to balance distributional fidelity and decoding efficiency at each token position.

Thus, $\{\eta_j\}_{j \geq 1}$ determines the trajectory of the correction level. We define three correction schedules $\{\eta_j\}_{j \geq 1}$, motivated by the empirical analysis in Section 6.2.

(i) *Step schedule.* The step schedule keeps the correction level at its initial value before position L and switches directly to the final value at position L :

$$\eta_{\text{step}}(j) = \mathbf{1}[j > L] = \begin{cases} 0, & j \leq L, \\ 1, & j > L. \end{cases} \quad (12)$$

This schedule makes an abrupt transition between the two boundary correction levels at the switch position L .

(ii) *Linear schedule.* The linear schedule interpolates between the two boundary correction levels at a constant rate until the terminal value is reached:

$$\eta_{\text{linear}}(j) = \text{clip}\left(\frac{j}{L+1}, 0, 1\right). \quad (13)$$

This schedule changes η_j linearly, with larger L producing a slower transition and smaller L producing a faster one.

(iii) *Power-law schedule.* The power-law schedule generalizes the linear transition with a curvature parameter $\gamma > 0$:

$$\eta_\gamma(j) = \left[\text{clip}\left(\frac{j}{L+1}, 0, 1\right)\right]^\gamma. \quad (14)$$

For $\gamma = 1$, the power-law schedule reduces to the linear schedule. Values $\gamma > 1$ delay the transition toward later positions, while values $0 < \gamma < 1$ make it occur earlier.

Takeaway 2. SECURESD uses the verification correction schedule to control how strongly each decoding position follows the standard verifier or the relaxed verifier. This behavior is determined by the correction schedule $\{\eta_j\}_{j \geq 1}$, which allows SECURESD to balance utility and efficiency without changing the verifier itself.

6. Why SECURESD Works: Analysis

This section provides theoretical and empirical analyses of how SECURESD mitigates performance degradation under lossy speculative decoding. The theoretical analysis characterizes the task-agnostic performance-retention benefit of verifier correction, while the empirical analysis identifies the greater early-position sensitivity of security metrics. This distinction explains why SECURESD applies stronger correction to early tokens on security tasks.

6.1. Theoretical Analysis of SECURESD

Output Distribution. A speculative decoding step at position t proceeds as follows. Conditioned on the prefix $x_{<t}$, the draft model first proposes a token $\tilde{x}_t \sim p_t$. The proposed token is then accepted with probability $\alpha_t(\tilde{x}_t)$. If it is accepted, the system sets $x_t = \tilde{x}_t$; otherwise, the system samples a replacement token from the recovery distribution $x_t \sim R_t$. Equivalently, the one-step transition of the system can be written as

$$x_t = \begin{cases} \tilde{x}_t, & \text{with probability } \alpha_t(\tilde{x}_t), \\ \hat{x}_t, \hat{x}_t \sim R_t, & \text{with probability } 1 - \alpha_t(\tilde{x}_t). \end{cases} \quad (15)$$

Through this definition, we can derive the output distribution of speculative decoding with the relaxed verifier.

Theorem 1 (Output Distribution of Relaxed Verifiers). *Fix a prefix $x_{<t}$ and consider a relaxed speculative decoding verifier $S(q, p, \alpha, R)$. Let $\tilde{q}_t$ denote the output distribution induced by one decoding step of S , i.e.,*

$$\tilde{q}_t(x) \triangleq \Pr_S(x_t = x \mid x_{<t}). \quad (16)$$

Then, for every token $x \in \mathcal{V}$, $\tilde{q}_t(x)$ is given by

$$\tilde{q}_t(x) = p_t(x)\alpha_t(x) + (1 - \mathbb{E}_{u \sim p_t}[\alpha_t(u)]) R_t(x). \quad (17)$$

The proof is provided in Appendix A.1.

Corollary 1 (Output Distribution of Standard SD). *For a fixed prefix $x_{<t}$, consider the standard SD verifier with acceptance function and recovery distribution defined as*

$$\alpha_t^{\text{sd}}(x) = \min\left(1, \frac{q_t(x)}{p_t(x)}\right), \quad (18)$$

$$R_t^{\text{sd}}(x) = \frac{(q_t(x) - p_t(x))_+}{\sum_{u \in \mathcal{V}} (q_t(u) - p_t(u))_+}, \quad (19)$$

where $(f)_+ = \max(0, f)$. Then the one-step output distribution induced by the verifier matches the target distribution:

$$\tilde{q}_t^{\text{sd}}(x) = q_t(x), \quad \forall x \in \mathcal{V}. \quad (20)$$

The proof is provided in Appendix A.2.

Corollary 1 establishes that the standard verifier is a zero-deviation baseline for the subsequent analysis.

Distributional Deviation. We then measure the one-step distortion of a relaxed verifier by the total variation distance $\text{TV}(\tilde{q}_t, q_t)$ between its output $\tilde{q}_t$ and the target q_t . The following theorem decomposes $\text{TV}(\tilde{q}_t, q_t)$ into two sources corresponding to the two design choices of the verifier.

Theorem 2 (Decomposition of Distributional Deviation). *For any relaxed verifier $S(q, p, \alpha, R)$ at draft position t , define the pointwise acceptance deviation $\delta_t(x) \triangleq \alpha_t(x) - \alpha_t^{\text{sd}}(x)$. It follows that*

$$\begin{aligned} \text{TV}(\tilde{q}_t, q_t) &\leq \underbrace{\mathbb{E}_{x \sim p_t} [|\delta_t(x)|]}_{\text{acceptance mismatch}} \\ &\quad + \underbrace{(1 - \mathbb{E}_{x \sim p_t} [\alpha_t(x)]) \text{TV}(R_t, R_t^{\text{sd}})}_{\text{recovery mismatch}}. \end{aligned} \quad (21)$$

The proof is provided in Appendix A.3.

Takeaway 3. Theorem 2 attributes the distributional deviation to two factors: (1) the acceptance bias introduced by the relaxed verifier, and (2) the discrepancy between the recovery distribution and the standard recovery distribution. This observation suggests that SECURESD requires a coordinated design for correcting both α and R .

Performance Degradation Decomposition. We connect the distributional deviation $\text{TV}(\tilde{q}_t, q_t)$ to downstream task metrics. Let $S_m : \mathcal{V}^T \rightarrow [0, 1]$ be the scoring function for the specific task m , and let

$$M = S_m(x_{1:T}) \in [0, 1] \quad (22)$$

denote the task score. Write $\tilde{Q}$ (resp. Q) for the full-sequence distribution induced by the relaxed verifier (resp. standard verifier). For a fixed prefix $x_{<t}$, define

$$\mu_t(x; x_{<t}) \triangleq \mathbb{E}_Q[M \mid x_{<t}, x_t = x], \quad (23)$$

as the expected task score when token x is chosen at position t and the remaining tokens are drawn from q .

Theorem 3 (Task Metric Drift Bound). *For any task metric $M \in [0, 1]$, let $\tilde{Q}_{<t}$ be the marginal of $\tilde{Q}$ over $x_{<t}$, write $\text{osc}(f) \triangleq \max_x f(x) - \min_x f(x)$, and write $\text{TV}_t(x_{<t})$ for the distributional deviation $\text{TV}(\tilde{q}_t, q_t)$ of Theorem 2, viewed as a function of the prefix $x_{<t}$. Define*

$$\Gamma \triangleq \sum_{t=1}^T \mathbb{E}_{x_{<t} \sim \tilde{Q}_{<t}} [\text{TV}_t(x_{<t}) \cdot \text{osc}(\mu_t(\cdot; x_{<t}))]. \quad (24)$$

With Γ as defined in Equation (24), the metric gap satisfies

$$|\mathbb{E}_{\tilde{Q}}[M] - \mathbb{E}_Q[M]| \leq \min\{\Gamma, 1\}. \quad (25)$$

Letting $\text{TV}_t \triangleq \sup_{x_{<t}} \text{TV}_t(x_{<t})$ denote the worst-case distributional deviation and $\text{osc}(\mu_t) \triangleq \sup_{x_{<t}} \text{osc}(\mu_t(\cdot; x_{<t}))$ denote the worst-case task-score oscillation, we have

$$|\mathbb{E}_{\tilde{Q}}[M] - \mathbb{E}_Q[M]| \leq \min\left\{\sum_{t=1}^T \text{TV}_t \cdot \text{osc}(\mu_t), 1\right\}. \quad (26)$$

The proof is provided in Appendix A.4.

Theorem 3 links performance degradation to the distributional deviation of the relaxed verifier. Notably, the bound depends not only on the deviation between $\tilde{q}_t$ and q_t , but also on the task sensitivity to the token chosen at position t . This sensitivity is captured by $\text{osc}(\mu_t(\cdot; x_{<t}))$, which measures the largest possible change in the final metric induced by a one-token decision. We formalize it as follows.

Definition 2 (Positional Task Sensitivity). *The positional task sensitivity at position t given prefix $x_{<t}$ is*

$$\begin{aligned} I_t(x_{<t}) &\triangleq \text{osc}(\mu_t(\cdot; x_{<t})) \\ &= \max_{x \in \mathcal{V}} \mu_t(x; x_{<t}) - \min_{x \in \mathcal{V}} \mu_t(x; x_{<t}). \end{aligned} \quad (27)$$

TABLE 4: **Magnitude of ϵ_t relative to $\eta_t \text{TV}_t$.** Across four datasets, ϵ_t is negligible compared to $\eta_t \text{TV}_t$.

Method	$\eta = 0.25$	$\eta = 0.5$	$\eta = 0.75$
LossySD	0.00%	0.00%	0.00%
FSD	1.03%	0.67%	0.33%
MARS	0.48%	0.32%	0.16%
BiLD	0.62%	0.41%	0.21%
FLy	2.55%	1.70%	0.85%
SpecCascade	0.00%	0.00%	0.00%

Takeaway 4. Theorem 3 shows that performance degradation introduced by the relaxed verifier can be decomposed into: (1) the distributional deviation $\text{TV}(\tilde{q}_t, q_t)$, and (2) the task’s sensitivity I_t to token choices at each position.

Distributional Deviation of SECURESD. We now analyze how SECURESD reduces the distributional deviation. Recall from Algorithm 2 that at position t , SECURESD applies the mixed acceptance function and recovery distribution

$$\alpha_t^\eta = (1 - \eta_t)\alpha_t^{\text{sd}} + \eta_t\alpha_t, \quad (28)$$

$$R_t^\eta = (1 - \eta_t)R_t^{\text{sd}} + \eta_t R_t. \quad (29)$$

Applying Theorem 1 to the mixed verifier $S(q, p, \alpha^\eta, R^\eta)$ gives the output distribution:

$$\tilde{q}_t^\eta(x) = p_t(x)\alpha_t^\eta(x) + (1 - \bar{\alpha}_t^\eta)R_t^\eta(x), \quad (30)$$

where $\bar{\alpha}_t^\eta \triangleq \mathbb{E}_{u \sim p_t}[\alpha_t^\eta(u)] = (1 - \eta_t)\bar{\alpha}_t^{\text{sd}} + \eta_t \bar{\alpha}_t$.

Proposition 1 (Degradation Mitigation via SECURESD). *Under $\eta_t \in [0, 1]$ at position t , the distributional deviation of the SECURESD verifier satisfies*

$$\text{TV}(\tilde{q}_t^\eta, q_t) \leq \eta_t \text{TV}(\tilde{q}_t, q_t) + \eta_t(1 - \eta_t)|\bar{\alpha}_t - \bar{\alpha}_t^{\text{sd}}| \text{TV}(R_t, R_t^{\text{sd}}). \quad (31)$$

Let $\epsilon_t = \eta_t(1 - \eta_t)|\bar{\alpha}_t - \bar{\alpha}_t^{\text{sd}}| \text{TV}(R_t, R_t^{\text{sd}})$. The task metric drift under SECURESD satisfies

$$\left| \mathbb{E}_{\tilde{Q}^\eta}[M] - \mathbb{E}_Q[M] \right| \leq \min \left\{ \sum_{t=1}^T (\eta_t \text{TV}_t + \epsilon_t) I_t, 1 \right\}, \quad (32)$$

where TV_t , ϵ_t , and I_t are uniform upper bounds over prefixes. The proof is provided in Appendix A.5.

Proposition 1 demonstrates that the performance degradation introduced by a relaxed verifier can be mitigated by reducing $\eta_t \text{TV}_t + \epsilon_t$. Importantly, we find that ϵ_t is typically zero, and even when non-zero, it is negligible compared to the dominant term $\eta_t \text{TV}_t$. For instance, under the STEP schedule, $\epsilon_t = 0$ because $\eta_t \in \{0, 1\}$; similarly, in LossySD, $\epsilon_t = 0$ because $R_t = R_t^{\text{sd}}$. For cases where $\epsilon_t \neq 0$, we evaluate its magnitude relative to $\eta_t \text{TV}_t$ in Table 4. The ratio of ϵ_t to $\eta_t \text{TV}_t$ ranges from 0% to 2.55%, averaging only 0.52%. This indicates that the bound is dominated by $\eta_t \text{TV}_t$ and thus scales approximately linearly with η_t . We observe a consistent empirical trend in Figure 4.

Takeaway 5. Theorem 3 relates the performance degradation under lossy speculative decoding to both the output distribution shift induced by the relaxed verifier and the

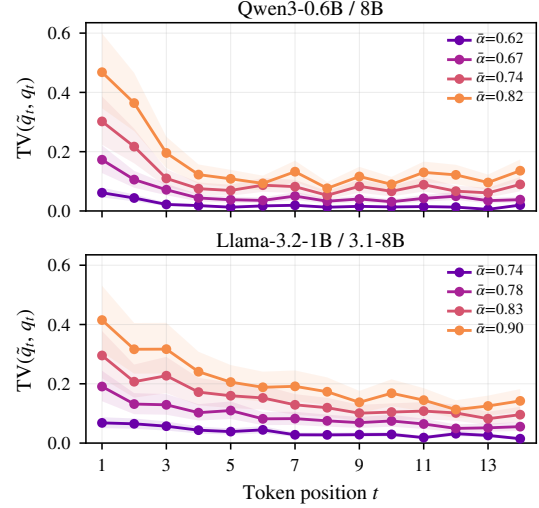


Figure 2: **Distributional Deviation of Relaxed Verifiers.** Per-position deviation $\text{TV}(\tilde{q}_t, q_t)$ under relaxed verifiers concentrates at early token positions and is amplified by higher acceptance rates.

task type. Both Proposition 1 and our empirical analysis demonstrate that SECURESD can reduce $\eta_t \text{TV}_t$ through correction to mitigate performance degradation.

6.2. Empirical Study of Theoretical Analysis

The analysis in Theorem 3 attributes the performance degradation of relaxed SD to two factors. The first is the per-step distributional deviation $\text{TV}(\tilde{q}_t, q_t)$ characterized in Theorem 2, and the second is the positional task sensitivity I_t defined in Definition 2. It further predicts that η -mixing linearly contracts the deviation, as shown in Proposition 1. We now empirically validate these three predictions.

Distributional Deviation of Relaxed Verifiers. We estimate the per-position deviation $\text{TV}(\tilde{q}_t, q_t)$ on both security and utility benchmarks in Figure 2. The deviation shows a clear positional pattern: it is concentrated at early generation positions and rapidly diminishes as decoding proceeds. As the acceptance rate increases, the deviation becomes larger, suggesting that more permissive verifiers induce stronger departures from the target distribution. This amplification is most evident at early positions, where deviations can propagate through the autoregressive decoding process.

Positional Task Sensitivity. Computing the task sensitivity I_t exactly is intractable, since it requires evaluating $\mu_t(x; x_{<t})$ for every token $x \in \mathcal{V}$. We therefore estimate it with Monte Carlo sampling. For each sampled position t , we select the top- k candidate tokens under q_t , generate N target-model continuations for each candidate, and score the resulting completions with the task metric. We then use the range of the average scores as the estimate $\hat{I}_t$.

Figure 3 shows how the positional task sensitivity I_t changes over the course of generation. We find that security and utility benchmarks exhibit different I_t patterns.

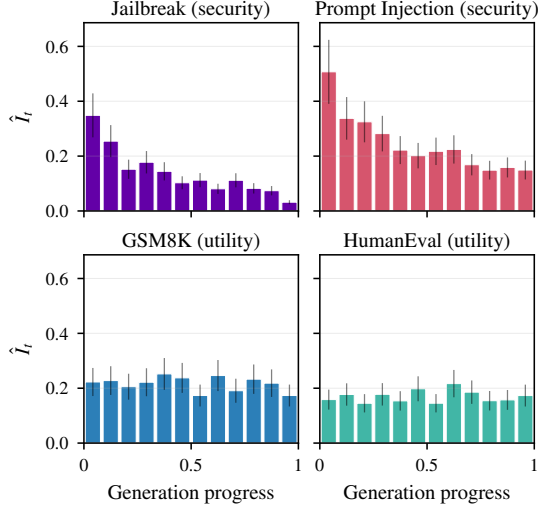


Figure 3: **Positional Task Sensitivity.** $\hat{I}_t$ measures the task sensitivity to tokens at different positions. Security benchmarks show early concentrated $\hat{I}_t$, while utility benchmarks show a more uniform distribution across generation.

For security benchmarks, I_t is mostly concentrated in the early generated tokens, whereas for utility benchmarks, I_t is more evenly distributed across the generation process. This difference arises because utility-oriented tasks, such as math and coding, often require step-by-step reasoning, where correctness depends on maintaining accuracy throughout the entire generation. In contrast, performance on security benchmarks is more strongly determined by the model’s early response behavior, such as whether it gives a safe or unsafe initial response and whether the generation is hijacked toward unintended content.

Takeaway 6. Proposition 1 and the following empirical analysis suggest that the performance degradation is mainly controlled by $\eta_t \cdot \text{TV}_t \cdot I_t$. On security tasks, both the distributional deviation and task sensitivity are concentrated in the first few tokens. This system-level result for draft-target model pairs aligns with prior findings on early-token sensitivity in single-model safety [46], [47]. Therefore, using smaller η_t at early positions can substantially mitigate the degradation. This observation motivates SECURESD in Algorithm 2.

Degradation Mitigation via SECURESD. We first validate the relationship between η and the distributional deviation predicted by Proposition 1. Figure 4 (left) plots the measured deviation $\text{TV}(\tilde{q}_t^\eta, q_t)$ as η varies from 1 to 0. The deviation decreases nearly linearly with η , closely matching the contraction bound in Proposition 1.

We next examine whether applying η -verification correction to the first few tokens can recover model performance. Figure 4 (right) studies a representative setting where utility PRR remains high but security PRR drops substantially. Under the step correction schedule, security PRR increases with L and gradually saturates. This trend matches the

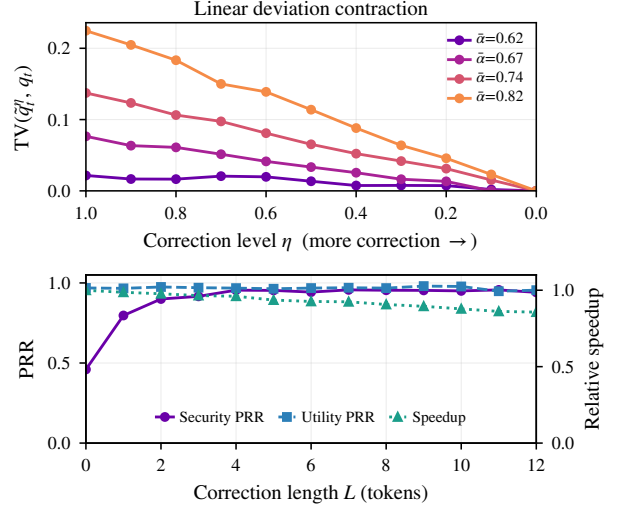


Figure 4: **Degradation Mitigation via SECURESD.** *Left:* the measured deviation $\text{TV}(\tilde{q}_t^\eta, q_t)$ contracts linearly in η . *Right:* security PRR shows a saturating increase as L grows.

positional pattern of $\text{TV}_t \cdot I_t$ on security tasks and supports our theoretical analysis. Large L values, however, gradually reduce SPD by introducing more strict verification steps. In practice, SECURESD should choose the correction schedule according to the desired PRR–SPD trade-off.

Takeaway 7. Both theoretical and empirical analyses support the framework in three key aspects: relaxed verification induces distributional deviation, security tasks are highly sensitive to early-token errors, and η -mixing contracts deviation as predicted. These findings justify SECURESD: enforcing stronger early correction for security while relaxing later verification for efficiency.

7. Experiments

In this section, we conduct a comprehensive evaluation of SECURESD to assess its ability to improve security while preserving utility and efficiency. We also design extensive ablation studies to demonstrate that SECURESD can be applied to LLM systems with different parameter scales while remaining effective.

7.1. Experimental Setup

We follow the same experimental setup as in Section 4.2 and introduce two additional metrics to quantify the performance gains of SECURESD. We use the step schedule with $L = 1$ as the default setting of SECURESD.

Performance Retention Rate Gain Δ_{PRR} . Δ_{PRR} measures how much additional target-model performance SECURESD retains on top of a relaxed verifier. To isolate the effect of SECURESD, we compare in a *paired* manner: the two runs use the same verifier with the same hyperparameter

TABLE 5: **Impact of SECURESD on LLM System Safety/Security/Utility/Efficiency.** We evaluate SECURESD (default) with six relaxed/lossy verifiers on Qwen3-0.6B/8B and Llama3-1B/8B across four datasets. The average Δ_{PRR} and Δ_{SPD} show that SECURESD improves security and safety while largely maintaining target-model utility and lossy SD efficiency.

Relaxed Verifier	Model Family	Safety/Security Benchmarks				Utility Benchmarks			
		Jailbreak-SD		OpenPromptInjection		HumanEval		GSM8K	
		Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}
LossySD [1]	Qwen3-0.6B/8B	+0.337	+0.050	+0.125	-0.010	-0.013	+0.001	+0.025	-0.020
	Llama3-1B/8B	+0.045	+0.003	+0.071	-0.002	+0.025	-0.002	+0.010	-0.005
BiLD [22]	Qwen3-0.6B/8B	+0.295	+0.063	+0.086	-0.010	-0.054	-0.016	+0.026	-0.010
	Llama3-1B/8B	+0.039	+0.018	+0.073	-0.005	-0.002	+0.001	.000	-0.004
SpecCascade [23]	Qwen3-0.6B/8B	+0.598	+0.083	+0.229	-0.016	+0.016	-0.005	+0.196	+0.006
	Llama3-1B/8B	+0.074	+0.037	+0.197	+0.001	+0.022	+0.004	+0.091	+0.008
FSD [24]	Qwen3-0.6B/8B	+0.122	-0.023	+0.084	-0.016	-0.007	-0.004	+0.007	-0.004
	Llama3-1B/8B	+0.051	+0.020	+0.122	+0.003	+0.008	-0.001	+0.002	-0.002
FLy [25]	Qwen3-0.6B/8B	+0.194	-0.038	+0.152	-0.018	-0.022	-0.006	+0.003	-0.005
	Llama3-1B/8B	+0.007	+0.002	+0.095	-0.007	+0.035	-0.001	+0.004	-0.003
MARS [26]	Qwen3-0.6B/8B	+0.072	+0.021	+0.009	-0.014	+0.034	-0.017	+0.018	-0.012
	Llama3-1B/8B	+0.036	+0.032	+0.038	+0.001	+0.018	+0.002	-0.014	-0.003
Average Lossy SD	Qwen3-0.6B/8B	+0.270	+0.026	+0.114	-0.014	-0.008	-0.008	+0.046	-0.007
	Llama3-1B/8B	+0.042	+0.019	+0.099	-0.001	+0.018	+0.001	+0.015	-0.001

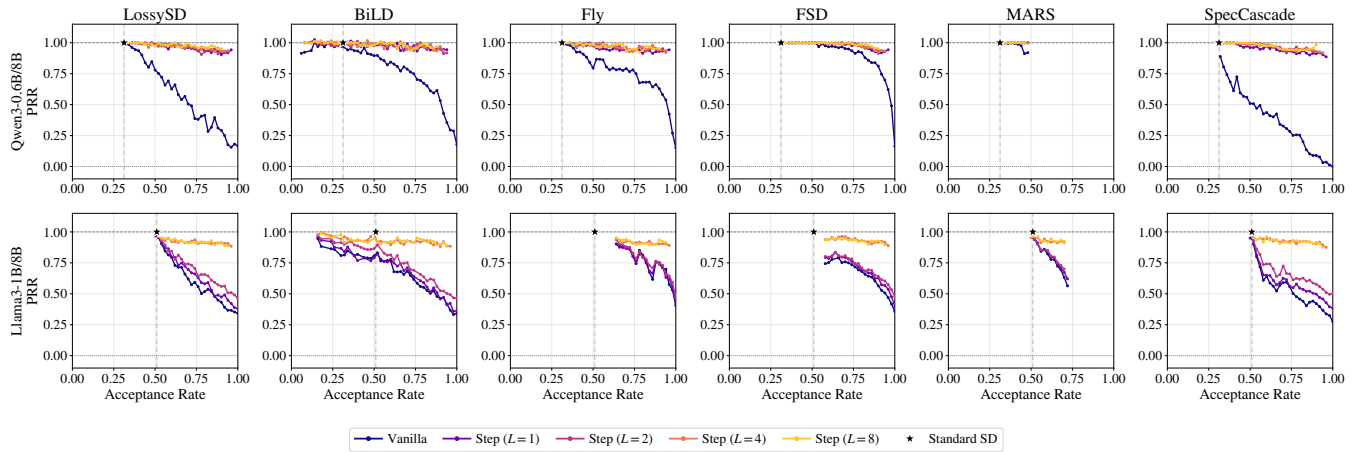


Figure 5: **Security PRR versus acceptance rate on Jailbreak-SD.** Columns are relaxed verifiers and rows are model families (Qwen3-0.6B/8B, Llama3-1B/8B). The star marks lossless speculative decoding. As the AR increases, vanilla PRR degrades sharply, while SECURESD (STEP (L)) keeps it near the lossless level.

θ , and differ only in whether SECURESD is applied. Let $\text{PRR}_{\text{SECURESD}}^{\theta}$ and $\text{PRR}_{\text{vanilla}}^{\theta}$ denote the PRRs of the verifier with and without SECURESD under hyperparameter θ . We define

$$\Delta_{\text{PRR}} = \frac{1}{|\Theta|} \sum_{\theta \in \Theta} \left(\text{PRR}_{\text{SECURESD}}^{\theta} - \text{PRR}_{\text{vanilla}}^{\theta} \right), \quad (33)$$

where Θ is the hyperparameter set of the verifier. A positive Δ_{PRR} indicates that SECURESD retains more target model performance than the vanilla verifier.

Inference Speedup Gain Δ_{SPD} . Δ_{SPD} quantifies the efficiency benefit of applying SECURESD under the same paired comparison. Let $\text{SPD}_{\text{SECURESD}}^{\theta}$ and $\text{SPD}_{\text{vanilla}}^{\theta}$ denote

the inference speedups (SPD) of the verifier with and without SECURESD under the same hyperparameter θ . We define

$$\Delta_{\text{SPD}} = \frac{1}{|\Theta|} \sum_{\theta \in \Theta} \left(\text{SPD}_{\text{SECURESD}}^{\theta} - \text{SPD}_{\text{vanilla}}^{\theta} \right). \quad (34)$$

A small negative Δ_{SPD} indicates that SECURESD introduces a minor efficiency overhead, while a positive value indicates that SECURESD is faster than the vanilla verifier.

7.2. Main Results

Table 5 summarizes the paired improvements of SECURESD, measured by Δ_{PRR} and Δ_{SPD} , over six relaxed SD verifiers on one safety, two security, and two utility

TABLE 6: **Impact of the correction length L on SECURESD.** Δ_{PRR} increases consistently with L , indicating performance gains from correcting the generation of more tokens, especially on Llama3-1B/8B. Δ_{SPD} changes only slightly as L increases, indicating that this behavior incurs only a minor efficiency overhead. Results are averaged across six lossy-SD methods.

L	Qwen3-0.6B/8B								Llama3-1B/8B							
	Jailbreak-SD		Prompt Inj.		HumanEval		GSM8K		Jailbreak-SD		Prompt Inj.		HumanEval		GSM8K	
	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}
1	+0.270	+0.026	+0.114	-0.014	-0.008	-0.008	+0.046	-0.007	+0.042	+0.019	+0.099	-0.001	+0.018	+0.001	+0.015	-0.001
2	+0.279	+0.006	+0.152	-0.021	-0.009	-0.009	+0.045	-0.009	+0.096	+0.014	+0.109	-0.011	+0.023	+0.000	+0.021	-0.011
4	+0.282	.000	+0.213	-0.026	-0.008	-0.015	+0.032	-0.018	+0.321	-0.008	+0.140	-0.022	+0.023	-0.001	+0.027	-0.005
8	+0.285	-0.047	+0.224	-0.035	-0.005	-0.023	+0.034	-0.035	+0.325	-0.052	+0.135	-0.035	+0.027	-0.010	+0.033	-0.019

TABLE 7: **Evaluation of SECURESD on AgentDojo.** We evaluate SECURESD with Qwen3-0.6B/8B on the realistic, long-context prompt injection benchmark AgentDojo. The *utility under attack* of the target and draft models is .264 and .078, respectively. SECURESD improves the average PRR while preserving SPD.

Method	w/o SECURESD		w/ SECURESD		$\Delta_{\text{PRR}} \uparrow$	$\Delta_{\text{SPD}} \uparrow$
	PRR $\uparrow$	SPD $\uparrow$	PRR $\uparrow$	SPD $\uparrow$		
LossySD	.548	1.919	.723	1.947	+0.175	+0.028
BiLD	.932	1.872	.983	1.891	+0.051	+0.019
SpecCascade	.169	2.081	.328	2.158	+0.159	+0.077
FSD	.869	1.839	.955	1.847	+0.086	+0.007
FLY	.793	1.798	.904	1.799	+0.111	+0.001
MARS	.863	1.882	.953	1.910	+0.090	+0.028
Average	.696	1.898	.808	1.925	+0.112	+0.027

benchmarks. Figure 5 further reports the safety PRR on the Jailbreak benchmark as a function of the acceptance rate. Table 7 shows the security results of evaluating SECURESD on AgentDojo. For each verifier and model family, we compare standard lossless SD, the vanilla relaxed verifier, and the corresponding verifier augmented with SECURESD.

SECURESD Mitigates Safety and Security Degradation. Across most verifier-model combinations, SECURESD yields positive gains. For Qwen3-0.6B/8B, the average Δ_{PRR} reaches +0.270 on Jailbreak and +0.114 on Prompt Injection. The largest single improvement is observed for SpecCascade, where SECURESD improves PRR by up to +0.598. Figure 5 further shows that vanilla relaxed verifiers lose PRR rapidly as the acceptance rate increases, whereas SECURESD keeps PRR close to that of the standard verifier. This confirms that correcting early accepted positions recovers most of the lost safety/security performance while retaining high acceptance rates.

SECURESD Preserves Utility Performance. Unlike the substantial gains on safety/security benchmarks, SECURESD has little effect on utility benchmarks. For Qwen3-0.6B/8B, the average Δ_{PRR} is -0.008 on HumanEval and +0.046 on GSM8K. These small changes show that early-position correction does not materially hurt task utility, consistent with our finding that utility-oriented tasks are less sensitive to the deviations corrected by SECURESD. In some cases, corrected prefixes even improve utility, such as the +0.196 PRR gain of SpecCascade on GSM8K.

SECURESD Introduces Negligible Efficiency Overhead. SECURESD has only a minor impact on decoding efficiency, with Δ_{SPD} remaining within ± 0.02 across most settings. Stricter early verification may reject more draft tokens, but the corrected prefixes can improve later acceptance rates, making the overall speed impact nearly neutral. In some cases, SECURESD is even faster than the vanilla verifier. For example, SpecCascade gains +0.083 in speedup on Jailbreak with Qwen3-0.6B/8B, with an average gain of +0.026 in this setting.

7.3. Effect of the Correction Length L

Table 6 reports the effect of varying the step correction length $L \in \{1, 2, 4, 8\}$ on Δ_{PRR} and Δ_{SPD} . The two model families exhibit different sensitivities to L . For Qwen3-0.6B/8B, the safety gain on Jailbreak is already close to saturation at $L = 1$, with the average Δ_{PRR} increasing only slightly from +0.270 at $L = 1$ to +0.285 at $L = 8$. In contrast, Llama3-1B/8B benefits substantially from a longer correction window. Its average Δ_{PRR} on Jailbreak increases from +0.042 at $L = 1$ to +0.321 at $L = 4$, and then plateaus at +0.325 when $L = 8$.

Figure 5 shows the same trend across acceptance rates. For Qwen3, all Step curves separate clearly from the vanilla relaxed verifier even with $L = 1$, and larger L brings only marginal additional improvement. For Llama3, however, the Step curves move away from the vanilla verifier progressively as L increases, indicating that correcting only the first accepted position is insufficient for the Llama3 family.

This difference is consistent with the distributional-deviation measurements in Figure 2. For Qwen3, the per-position deviation $\text{TV}(\tilde{q}_t, q_t)$ drops rapidly after the first few positions, suggesting that most harmful deviation is concentrated at the beginning of the generated response. Therefore, correcting the first accepted position already restores most of the target-consistent behavior. For Llama3, the deviation decays more slowly across positions, so relaxed verification can continue to steer the committed prefix away from the target model several tokens after the first acceptance. A longer correction window is thus needed to prevent this drift.

Increasing L also introduces a mild efficiency cost. On Jailbreak, the average Δ_{SPD} decreases from +0.026 at $L = 1$ to -0.047 at $L = 8$ for Qwen3, and from +0.019 to -0.052 for Llama3. Overall, L provides a controllable trade-off between safety recovery and decoding speed: $L = 1$ is

TABLE 8: **Impact of the correction schedule η on SECURESD.** S/L/P denote the STEP/LINEAR/POWER-LAW schedules, respectively. All three schedules effectively improve PRR. In practice, STEP avoids complex distribution-mixing operations, giving it an advantage in preserving SPD.

η Schedule	Jailbreak-SD				HumanEval			
	Qwen3		Llama3		Qwen3		Llama3	
	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}
S ($L=1$)	+0.337	+0.050	+0.045	+0.003	-0.013	+0.001	+0.025	-0.002
S ($L=4$)	+0.349	+0.023	+0.292	-0.001	-0.004	-0.006	+0.035	-0.004
L ($L=1$)	+0.158	-0.039	+0.019	-0.010	-0.008	-0.012	+0.027	-0.010
L ($L=8$)	+0.313	-0.111	+0.207	-0.110	-0.009	-0.014	+0.037	-0.017
P ($\gamma=2, L=1$)	+0.248	-0.065	+0.027	-0.015	+0.005	-0.014	+0.026	-0.009
P ($\gamma=2, L=8$)	+0.349	-0.148	+0.266	-0.119	-0.015	-0.020	+0.033	-0.014
P ($\gamma=4, L=1$)	+0.313	-0.111	+0.036	-0.019	-0.007	-0.021	+0.034	-0.011
P ($\gamma=4, L=8$)	+0.347	-0.158	+0.291	-0.186	+0.012	-0.030	+0.038	-0.015

sufficient for Qwen3, whereas $L = 4$ recovers nearly all safety gains for Llama3 with only a small average speed change of -0.008 .

How to Select L . Table 6 suggests $L \leq 8$ as an empirical choice across tasks. Conceptually, the choice of L should be related to the depth of safety alignment in the model [46]. When L matches the effective depth of safety alignment, the system can ensure that generation begins from a distribution consistent with the model’s safety-aligned behavior. A more robust approach is to perform an ablation study over L on a small calibration dataset and obtain curves similar to those in Figure 4, characterizing the trade-offs among Security/Safety-PRR, Utility-PRR, and SPD. One can then select L according to the requirements of the deployment setting. When no representative calibration dataset is available, using a relatively large L is a safer choice, since increasing L has only a limited impact on the overall decoding speedup and is generally worthwhile given the corresponding improvement in system-level performance.

7.4. Effect of Correction Schedule $\{\eta_j\}_{j \geq 1}$

The experiments above use the step schedule, which applies full correction to the first L positions and then switches directly to the relaxed verifier. A natural question is whether a smoother transition would be preferable. Table 8 studies this on LossySD by replacing the step schedule with linear and power-law schedules, varying the transition length $L \in \{1, 8\}$ and the curvature $\gamma \in \{2, 4\}$, and comparing against the step schedule at $L \in \{1, 4\}$, under the same paired protocol on Jailbreak and HumanEval. The smooth schedules reproduce the qualitative behavior of the step schedule: security gains increase as the transition becomes longer and more delayed, reaching $+0.347$ on Qwen3 and $+0.291$ on Llama3 for the power-law schedule with $L = 8$ and $\gamma = 4$, while Δ_{PRR} on HumanEval stays flat across all configurations.

However, the step schedule provides a better security-efficiency trade-off. On Qwen3, the step schedule with $L =$

TABLE 9: **Evaluation on batched inference with larger target models.** We evaluate SECURESD with larger target models and report the average PRR and SPD across four datasets and six relaxed verifiers. SECURESD improves PRR while preserving SPD.

Benchmark	SECURESD	Qwen3-0.6B/32B		Llama3-1B/70B	
		PRR	SPD	PRR	SPD
Jailbreak-SD	w/o	0.727	1.985	0.571	3.738
	w/	0.973	1.752	0.690	3.947
OpenPromptInjection	w/o	0.652	1.853	0.512	3.153
	w/	0.855	1.882	0.956	3.161
HumanEval	w/o	0.653	2.392	0.500	3.943
	w/	0.627	2.538	0.667	3.874
GSM8K	w/o	0.820	2.637	0.791	3.785
	w/	0.843	2.497	0.791	3.737

1 matches the best power-law gain ($+0.337$ versus $+0.349$) while being faster ($+0.050$ versus -0.148 in speed); on Llama3, the step schedule with $L = 4$ matches it ($+0.292$ versus $+0.291$) with a much smaller speed cost (-0.001 versus -0.186). Intermediate correction levels are less efficient in our setting, as they extend correction beyond the most security-sensitive positions; the step schedule is also simpler to implement, since each position uses either standard or relaxed verification without materializing vocabulary-level mixtures. We therefore adopt the step schedule as the default configuration of SECURESD.

7.5. Ablation Study

We further examine whether the gains of SECURESD remain stable under different deployment and decoding configurations, varying the target-model scale, the decoding temperature τ , and the speculative length k . The τ and k ablations use the same paired protocol on the two security benchmarks with Qwen3-0.6B/8B, averaged over the six relaxed verifiers and their hyperparameter sweeps.

Batched Inference with Larger Target Models. Speculative decoding is often used in practice to accelerate larger models. In Table 9, we use Qwen3-32B and Llama3.3-70B-Instruct as target models and evaluate the improvements in PRR and SPD with and without SECURESD. We find that speculative decoding generally provides more pronounced speedups for larger target models. Meanwhile, SECURESD effectively improves the safety/security performance while preserving the acceleration benefits.

Decoding Temperature τ . Higher temperatures flatten the draft and target distributions and make the security-critical opening tokens more stochastic, which could plausibly affect SECURESD. Table 10 varies $\tau \in \{0, 0.3, 0.7, 1.0\}$ with $k = 5$ fixed. The security gain is largely insensitive to temperature: Δ_{PRR} stays within $+0.238$ to $+0.274$ on Jailbreak and remains consistently positive on Prompt Injection ($+0.114$ to $+0.200$), while Δ_{SPD} stays within ± 0.026 . The benefit of SECURESD is therefore not an artifact of greedy decoding and carries over to stochastic sampling.

TABLE 10: **Impact of the decoding temperature τ on SECURESD.** Although different temperatures introduce generation uncertainty and PRR/SPD fluctuations, SECURESD remains effective across all tested temperatures.

τ	Jailbreak-SD		OpenPromptInjection	
	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}
0	+0.270	+0.026	+0.114	-0.014
0.3	+0.269	+0.020	+0.155	-0.015
0.7	+0.238	+0.014	+0.200	-0.017
1.0	+0.274	+0.005	+0.168	-0.018

TABLE 11: **Impact of the speculative length k on SECURESD.** Different speculative lengths affect the generation process and acceptance rate, while SECURESD maintains stable safety/security recovery across all speculative lengths.

k	Jailbreak-SD		OpenPromptInjection	
	Δ_{PRR}	Δ_{SPD}	Δ_{PRR}	Δ_{SPD}
3	+0.266	+0.032	+0.114	-0.007
4	+0.261	+0.037	+0.113	-0.012
5	+0.270	+0.026	+0.114	-0.014
6	+0.268	+0.030	+0.113	-0.016
7	+0.270	+0.024	+0.128	-0.016
8	+0.277	+0.018	+0.118	-0.016

Speculative Length k . A larger k commits more draft tokens per verification round and could in principle amplify the draft model’s influence. Table 11 varies $k \in \{3, \dots, 8\}$ with $\tau = 0$ fixed and shows that this does not occur: Δ_{PRR} fluctuates only between +0.261 and +0.277 on Jailbreak and between +0.113 and +0.128 on Prompt Injection, while Δ_{SPD} remains small (−0.016 to +0.037). This stability follows by design: the correction schedule is defined over global token positions, so SECURESD always corrects the first L positions of the response regardless of how the speculation rounds are segmented.

8. Discussion

Adaptive Attacks. Since SECURESD corrects only the first L positions, an attacker aware of the defense could try to push the security-critical decision beyond the correction window. For example, the attacker could first elicit a benign-looking opening longer than L tokens and then steer the subsequent generation toward harmful content. We instantiate this attack with 200 prompt injections that first repeat the original question to delay malicious behavior beyond the correction window, using the Step schedule ($L = 12$). Table 12 shows that SECURESD increases AFR by 19 percentage points and PRR by 55.9–71.8 percentage points over the average of six lossy-SD methods, approaching target-model performance. For queries where the target succeeds but the draft fails, early correction creates a target-aligned prefix that increases subsequent draft-target disagreement, reducing draft-token acceptance beyond the correction window and preserving safety/security.

Runtime Overhead in Real-World Systems. Since SECURESD performs operations such as distribution fusion

TABLE 12: **Evaluation under adaptive attacks.** We evaluate SECURESD against adaptive prompt injections that delay malicious behavior beyond the correction window. By correcting early generation, SECURESD steers the response toward the target-model trajectory and remains effective against these adaptive attacks.

Setting	Qwen3-0.6B/8B		Llama3-1B/8B	
	AFR $\uparrow$	PRR $\uparrow$	AFR $\uparrow$	PRR $\uparrow$
Target	.625	–	.580	–
Draft	.285	–	.315	–
w/o SECURESD	.415	.382	.360	.168
w/ SECURESD	.605	.941	.550	.886
Δ	+0.190	+0.559	+0.190	+0.718

TABLE 13: **Runtime overhead of SECURESD.** We report relative inference-latency changes in our vLLM implementation, in units of $10^{-3}\%$. Positive values indicate overhead and negative values indicate speedup; overhead is negligible.

Method	Qwen3		Llama3	
	0.6B/8B	0.6B/32B	1B/8B	1B/70B
LossySD	+0.001	+0.001	+0.001	+0.001
BiLD	+0.001	+0.001	+0.001	+0.001
SpecCascade	-2.330	-0.583	-1.830	-0.227
FSD	+0.775	+0.194	+0.611	+0.076
FLy	+0.001	+0.001	+0.001	+0.001
MARS	+0.001	+0.001	+0.001	+0.001

over the first L tokens, it may introduce additional implementation overhead. We implement a lightweight SECURESD layer on vLLM and measure the time required to generate a fixed number of tokens across all datasets. Table 13 shows that the additional overhead remains below 0.001% because SECURESD reuses speculative-decoding intermediates and requires negligible computation relative to the target-model forward pass. For SpecCascade, SECURESD slightly improves speed by eliminating verifier-side recovery softmax and sampling. FSD incurs the largest positive overhead because it recomputes the full recovery distribution R , but the overhead remains negligible.

Early-Token Criticality for Safety and Security. Prior studies [46], [47] show that the first few generated tokens strongly influence the safety of a response. These studies examine intrinsic early-token sensitivity within a single model. We instead study system-level safety and security under lossy speculative decoding, where target-model decoding defines the performance upper bound, but relaxed verification can introduce early deviations from its generation trajectory. We show that these system-induced deviations degrade safety and security much more than utility, revealing a security–utility asymmetry not studied in prior work. This distinction motivates SECURESD, which restores stricter target verification at early positions.

9. Conclusion

In this paper, we presented the *first* systematic study of the security implications of lossy speculative decoding. We

revealed a clear safety/security–utility asymmetry, where relaxed verification improves efficiency but increases jailbreak and prompt injection risks much faster than it degrades utility. Our analysis attributed this asymmetry to early-position draft tokens, whose deviations from the target distribution shape the initial direction of the response. Based on this insight, we proposed SECURESD, a position-aware method that strengthens early verification while keeping efficient verification elsewhere. More broadly, our results show that inference optimizations can silently weaken model security, highlighting the need to include security evaluation in their utility and efficiency assessment.

Acknowledgments

We thank the reviewers and shepherd for their constructive comments on our work. The Authors acknowledge the National Artificial Intelligence Research Resource (NAIRR) Pilot for contributing to this research result. Neil Gong was supported in part by NSF under grant No. 2450935, 2112562, and 2625580.

References

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning*, 2023.
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating large language model decoding with speculative sampling,” *arXiv preprint arXiv:2302.01318*, 2023.
- [3] Y. Leviathan, M. Kalman, and Y. Matias, “Looking back at speculative decoding,” <https://research.google/blog/looking-back-at-speculative-decoding/>, 2024.
- [4] Google AI for Developers, “Speed-up gemma 4 with multi-token prediction,” <https://ai.google.dev/gemma/docs/mtp/overview>, 2026.
- [5] vLLM Project, “Speculative decoding,” https://docs.vllm.ai/en/latest/features/speculative_decoding/, 2026.
- [6] NVIDIA Triton Inference Server, “Speculative decoding with tensorrt-llm,” https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/tutorials/Feature_Guide/Speculative_Decoding/TRT-LLM/README.html, 2026.
- [7] Hugging Face Text Generation Inference, “Speculation,” <https://huggingface.co/docs/text-generation-inference/conceptual/speculation>, 2026.
- [8] SGLang Project, “Speculative decoding,” https://docs.sglang.io/docs/advanced_features/speculative_decoding, 2026.
- [9] Y. Zhou, K. Lyu, A. S. Rawat, A. K. Menon, A. Rostamizadeh, S. Kumar, J.-F. Kagy, and R. Agarwal, “Distillspec: Improving speculative decoding via knowledge distillation,” in *International Conference on Learning Representations*, 2024.
- [10] O. Zafrir, I. Margulis, D. Shteyman, S. Guskin, and G. Boudoukh, “Fastdraft: How to train your draft,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- [11] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang, “Online speculative decoding,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [12] Y. Hu, J. Guo, X. Feng, and T. Zhao, “Adaspec: Selective knowledge distillation for efficient speculative decoders,” *Advances in Neural Information Processing Systems*, 2025.
- [13] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-3: Scaling up inference acceleration of large language models via training-time test,” *Advances in Neural Information Processing Systems*, 2025.
- [14] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple llm inference acceleration framework with multiple decoding heads,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [15] Z. Ankner, R. Parthasarathy, A. Nrusimha, C. Rinard, J. Ragan-Kelley, and W. Brandon, “Hydra: Sequentially-dependent draft heads for medusa decoding,” in *First Conference on Language Modeling*, 2024.
- [16] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi *et al.*, “Specinfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024.
- [17] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-2: Faster inference of language models with dynamic draft trees,” in *Proceedings of the 2024 conference on empirical methods in natural language processing*, 2024.
- [18] G. Bachmann, S. Anagnostidis, A. Pumarola, M. Georgopoulos, A. Sanakoyeu, Y. Du, E. Schönfeld, A. Thabet, and J. Kohler, “Judge decoding: Faster speculative sampling requires going beyond model alignment,” in *ICLR 2025*, 2025.
- [19] K.-W. Lu, D.-Y. Hong, and P. Liu, “Adasd: Adaptive speculative decoding for efficient language model inference,” *arXiv preprint arXiv:2512.11280*, 2025.
- [20] J. Wang, Z. Tian, J. Li, Q. Xia, X. Duan, Z. Wang, B. Huai, and M. Zhang, “Alignment-augmented speculative decoding with alignment sampling and conditional verification,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [21] Z. Wang, S. R. Kasa, A. M. S, S. K. Kasa, J. Zou, S. Negi, R. Zhang, N. Jiang, and Q. Song, “Diversed: Relaxed speculative decoding via dynamic ensemble verification,” in *Proceedings of The 29th International Conference on Artificial Intelligence and Statistics*, 2026.
- [22] S. Kim, K. Mangalam, S. Moon, J. Malik, M. W. Mahoney, A. Ghلامي, and K. Keutzer, “Speculative decoding with big little decoder,” *Advances in Neural Information Processing Systems*, 2023.
- [23] H. Narasimhan, W. Jitkittum, A. S. Rawat, S. Kim, N. Gupta, A. K. Menon, and S. Kumar, “Faster cascades via speculative decoding,” in *International Conference on Learning Representations*, 2025.
- [24] M. Holsman, Y. Huang, and B. Dhingra, “Fuzzy speculative decoding for a tunable accuracy-runtime tradeoff,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- [25] J. Li, Y. Xu, G. Li, S. Yang, J. Xu, X. Yin, D. Li, E. Ngai, and E. Barsoum, “Training-free loosely speculative decoding: Accepting semantically correct drafts beyond exact match,” in *International Conference on Learning Representations*, 2026.
- [26] J. Song, X. Wang, H. Wang, X. Lei, T. Shi, S. Han, E. Yang, X.-W. Chang, and L. Ai, “Mars: Unleashing the power of speculative decoding via margin-aware verification,” in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026.
- [27] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint arXiv:2307.15043*, 2023.
- [28] L. Jiang, K. Rao, S. Han, A. Ettinger, F. Brahman, S. Kumar, N. Mireshghallah, X. Lu, M. Sap, Y. Choi *et al.*, “Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models,” *Advances in Neural Information Processing Systems*, 2024.

- [29] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Schweg, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramèr *et al.*, “Jailbreakbench: An open robustness benchmark for jailbreaking large language models,” *Advances in Neural Information Processing Systems*, 2024.
- [30] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li *et al.*, “Harmbench: a standardized evaluation framework for automated red teaming and robust refusal,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [31] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.
- [32] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, ““do anything now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024.
- [33] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and K. Wang, “A hitchhiker’s guide to jailbreaking chatgpt via prompt engineering,” in *Proceedings of the 4th International Workshop on Software Engineering and AI for Data Quality in Cyber-Physical Systems/Internet of Things*, 2024.
- [34] Y. Zeng, H. Lin, J. Zhang, D. Yang, R. Jia, and W. Shi, “How johnny can persuade llms to jailbreak them: Rethinking persuasion to challenge ai safety by humanizing llms,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [35] X. Liu, N. Xu, M. Chen, and C. Xiao, “Autodan: Generating stealthy jailbreak prompts on aligned large language models,” in *International Conference on Learning Representations*, 2024.
- [36] C. Anil, E. Durmus, N. Panickssery, M. Sharma, J. Benton, S. Kundu, J. Batson, M. Tong, J. Mu, D. Ford *et al.*, “Many-shot jailbreaking,” *Advances in Neural Information Processing Systems*, 2024.
- [37] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, “Jailbreaking black box large language models in twenty queries,” in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2025.
- [38] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 2023.
- [39] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” in *ML Safety Workshop, 36th Conference on Neural Information Processing Systems*, 2022.
- [40] R. Geng, C. Yin, Y. Wang, Y. Chen, and J. Jia, “Piarena: A platform for prompt injection evaluation,” *arXiv preprint arXiv:2604.08499*, 2026.
- [41] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [42] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [43] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents,” *Advances in neural information processing systems*, 2024.
- [44] A. Souly, Q. Lu, D. Bowen, T. Trinh, E. Hsieh, S. Pandey, P. Abbeel, J. Svegliato, S. Emmons, O. Watkins *et al.*, “A strongreject for empty jailbreaks,” *Advances in Neural Information Processing Systems*, 2024.
- [45] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu *et al.*, “A survey on llm-as-a-judge,” *The Innovation*, 2026.
- [46] X. Qi, A. Panda, K. Lyu, X. Ma, S. Roy, A. Beirami, P. Mittal, and P. Henderson, “Safety alignment should be made more than just a few tokens deep,” in *International Conference on Learning Representations*, 2025.
- [47] Z. Zhang, G. Shen, G. Tao, S. Cheng, and X. Zhang, “On large language models’ resilience to coercive interrogation,” in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.

Appendix A. Proofs of Theoretical Results

A.1. Proof of Theorem 1

Proof. Let A_t denote the event that the draft token at step t is accepted. Since $\tilde{x}_t \sim p_t$ and the verifier accepts token u with probability $\alpha_t(u)$, the overall acceptance rate is

$$\Pr_S(A_t \mid x_{<t}) = \sum_{u \in \mathcal{V}} p_t(u) \alpha_t(u) = \mathbb{E}_{u \sim p_t}[\alpha_t(u)]. \quad (35)$$

We expand $\tilde{q}_t(x)$ by conditioning on whether the draft token is accepted or rejected:

$$\begin{aligned} \tilde{q}_t(x) &= \Pr_S(x_t = x, A_t \mid x_{<t}) \\ &\quad + \Pr_S(A_t^c \mid x_{<t}) \Pr_S(x_t = x \mid A_t^c, x_{<t}). \end{aligned} \quad (36)$$

For the first term, using $\Pr_S(\tilde{x}_t = x \mid x_{<t}) = p_t(x)$ and $\Pr_S(A_t \mid \tilde{x}_t = x, x_{<t}) = \alpha_t(x)$:

$$\Pr_S(x_t = x, A_t \mid x_{<t}) = p_t(x) \alpha_t(x). \quad (37)$$

For the second term, rejection triggers sampling from R_t , so $\Pr_S(x_t = x \mid A_t^c, x_{<t}) = R_t(x)$. This gives

$$\tilde{q}_t(x) = p_t(x) \alpha_t(x) + (1 - \mathbb{E}_{u \sim p_t}[\alpha_t(u)]) R_t(x), \quad (38)$$

which completes the proof. $\square$

A.2. Proof of Corollary 1

Proof. Applying Theorem 1 with $\alpha_t = \alpha_t^{\text{sd}}$ and $R_t = R_t^{\text{sd}}$, the acceptance term evaluates to

$$p_t(x) \alpha_t^{\text{sd}}(x) = p_t(x) \min\left(1, \frac{q_t(x)}{p_t(x)}\right) \quad (39)$$

$$= \min(p_t(x), q_t(x)). \quad (40)$$

Since $\sum_u (p_t(u) - q_t(u)) = 0$, the positive and negative parts have equal total mass, so the rejection probability satisfies

$$1 - \mathbb{E}_{u \sim p_t}[\alpha_t^{\text{sd}}(u)] = \sum_{u \in \mathcal{V}} (p_t(u) - q_t(u))_+ \quad (41)$$

$$= \sum_{u \in \mathcal{V}} (q_t(u) - p_t(u))_+. \quad (42)$$

Substituting both into (17) gives

$$\tilde{q}_t^{\text{sd}}(x) = q_t(x), \quad (43)$$

which completes the proof. $\square$

A.3. Proof of Theorem 2

Proof. Let $\bar{\alpha}_t \triangleq \mathbb{E}_{u \sim p_t}[\alpha_t(u)]$ and $\bar{\alpha}_t^{\text{sd}} \triangleq \mathbb{E}_{u \sim p_t}[\alpha_t^{\text{sd}}(u)]$. Substituting $\tilde{q}_t$ from Theorem 1 and q_t from Corollary 1,

$$\text{TV}(\tilde{q}_t, q_t) = \frac{1}{2} \sum_{x \in \mathcal{V}} \left| p_t(x) \alpha_t(x) + (1 - \bar{\alpha}_t) R_t(x) - p_t(x) \alpha_t^{\text{sd}}(x) - (1 - \bar{\alpha}_t^{\text{sd}}) R_t^{\text{sd}}(x) \right|. \quad (44)$$

Using the definition $\delta_t(x) = \alpha_t(x) - \alpha_t^{\text{sd}}(x)$, which gives $\bar{\alpha}_t - \bar{\alpha}_t^{\text{sd}} = \mathbb{E}_{p_t}[\delta_t]$, we collect terms:

$$\text{TV}(\tilde{q}_t, q_t) = \frac{1}{2} \sum_{x \in \mathcal{V}} \left| p_t(x) \delta_t(x) - \mathbb{E}_{p_t}[\delta_t] R_t^{\text{sd}}(x) + (1 - \bar{\alpha}_t) (R_t(x) - R_t^{\text{sd}}(x)) \right|. \quad (45)$$

Applying the triangle inequality separates the contribution of the acceptance from that of the recovery, which yields

$$\text{TV}(\tilde{q}_t, q_t) \leq \frac{1}{2} \sum_{x \in \mathcal{V}} |p_t(x) \delta_t(x) - \mathbb{E}_{p_t}[\delta_t] R_t^{\text{sd}}(x)| + (1 - \bar{\alpha}_t) \text{TV}(R_t, R_t^{\text{sd}}). \quad (46)$$

For the first term, we apply the triangle inequality once more and use the condition $\sum_x R_t^{\text{sd}}(x) = 1$. This yields

$$\begin{aligned} & \frac{1}{2} \sum_{x \in \mathcal{V}} |p_t(x) \delta_t(x) - \mathbb{E}_{p_t}[\delta_t] R_t^{\text{sd}}(x)| \\ & \leq \frac{1}{2} \mathbb{E}_{p_t}[|\delta_t|] + \frac{1}{2} |\mathbb{E}_{p_t}[\delta_t]| \leq \mathbb{E}_{p_t}[|\delta_t|], \end{aligned} \quad (47)$$

where the last step uses $|\mathbb{E}_{p_t}[\delta_t]| \leq \mathbb{E}_{p_t}[|\delta_t|]$. Combining both bounds gives the stated inequality. $\square$

A.4. Proof of Theorem 3

Proof. Let $\text{TV}_t(x | x_{<t}) \triangleq \tilde{q}_t(x | x_{<t}) - q_t(x | x_{<t})$ be the signed pointwise deviation. Define hybrids $H^{(s)}$ using $\tilde{q}_t$ for $t \leq s$ and q_t for $t > s$, so $H^{(0)} = Q$ and $H^{(T)} = \tilde{Q}$. Telescoping over the hybrid chain yields:

$$\mathbb{E}_{\tilde{Q}}[M] - \mathbb{E}_Q[M] = \sum_{t=1}^T (\mathbb{E}_{H^{(t)}}[M] - \mathbb{E}_{H^{(t-1)}}[M]). \quad (48)$$

Since $H^{(t)}$ and $H^{(t-1)}$ differ only at position t and share the same prefix marginal $\tilde{Q}_{<t}$, the t -th term equals Δ_t , where

$$\Delta_t \triangleq \mathbb{E}_{x_{<t} \sim \tilde{Q}_{<t}} \left[\sum_{x \in \mathcal{V}} \text{TV}_t(x | x_{<t}) \mu_t(x; x_{<t}) \right]. \quad (49)$$

Hence $\mathbb{E}_{\tilde{Q}}[M] - \mathbb{E}_Q[M] = \sum_t \Delta_t$. For each fixed $x_{<t}$, the TV-oscillation inequality gives $|\sum_x \text{TV}_t(x | x_{<t}) \mu_t(x; x_{<t})| \leq \text{TV}_t(x_{<t}) \cdot \text{osc}(\mu_t(\cdot; x_{<t}))$. Taking expectations over $x_{<t} \sim \tilde{Q}_{<t}$,

$$|\Delta_t| \leq \mathbb{E}_{x_{<t} \sim \tilde{Q}_{<t}} [\text{TV}_t(x_{<t}) \cdot \text{osc}(\mu_t(\cdot; x_{<t}))]. \quad (50)$$

Summing over t and applying the triangle inequality gives $|\mathbb{E}_{\tilde{Q}}[M] - \mathbb{E}_Q[M]| \leq \Gamma$. The uniform bound (26) then follows by bounding $\text{TV}_t(x_{<t}) \leq \text{TV}_t$ and $\text{osc}(\mu_t(\cdot; x_{<t})) \leq \text{osc}(\mu_t)$ pointwise for each prefix $x_{<t}$. $\square$

A.5. Proof of Proposition 1

Proof. Expanding the definitions of α_t^η and R_t^η gives

$$\begin{aligned} \tilde{q}_t^\eta &= (1 - \eta_t) q_t + \eta_t \tilde{q}_t \\ &\quad + \eta_t (1 - \eta_t) (\bar{\alpha}_t - \bar{\alpha}_t^{\text{sd}}) (R_t - R_t^{\text{sd}}). \end{aligned} \quad (51)$$

Therefore, by the triangle inequality,

$$\begin{aligned} \text{TV}(\tilde{q}_t^\eta, q_t) &\leq \eta_t \text{TV}(\tilde{q}_t, q_t) \\ &\quad + \eta_t (1 - \eta_t) |\bar{\alpha}_t - \bar{\alpha}_t^{\text{sd}}| \text{TV}(R_t, R_t^{\text{sd}}). \end{aligned} \quad (52)$$

Taking uniform upper bounds over prefixes and substituting this result into Theorem 3 yields

$$|\mathbb{E}_{\tilde{Q}^\eta}[M] - \mathbb{E}_Q[M]| \leq \min \left\{ \sum_{t=1}^T (\eta_t \text{TV}_t + \epsilon_t) I_t, 1 \right\}. \quad (53)$$

$\square$

Appendix B. Lossless Baseline

Table 14 compares the PRR and SPD of lossless SD, average lossy SD, and SECURESD across four tasks and two model pairs.

TABLE 14: **Lossless baseline.** PRR and SPD are measured relative to target decoding. Lossy and SECURESD (STEP, $L = 1$) are averaged over six lossy-SD methods.

Task	PRR $\uparrow$			SPD $\uparrow$ ($\times$)		
	Lossless	Lossy	SECURESD	Lossless	Lossy	SECURESD
<i>Qwen3-0.6B/8B</i>						
Jailbreak	1.019	.782	1.052	1.049	1.543	1.569
Prompt Inj.	.915	.700	.814	1.057	1.439	1.425
HumanEval	.947	.874	.866	1.646	1.762	1.754
GSM8K	1.000	.932	.978	1.807	1.818	1.811
<i>Llama3-1B/8B</i>						
Jailbreak	.909	.682	.724	1.326	1.736	1.754
Prompt Inj.	.983	.755	.854	1.364	1.444	1.430
HumanEval	.933	.814	.832	1.849	1.854	1.855
GSM8K	.976	.915	.930	1.764	1.853	1.852

Appendix C.

Ethics Considerations

Stakeholder Analysis. Consider three stakeholder groups:

- *Model providers and infrastructure operators* who deploy LLM serving systems and develop serving frameworks (e.g., vLLM, TensorRT-LLM, and SGLang) that support lossy speculative decoding. Our findings reveal that relaxed verification, an efficiency optimization usually evaluated only on utility benchmarks, can substantially weaken jailbreak and prompt injection resilience. This can inform safer verification designs and the inclusion of security evaluation when tuning inference optimizations. The potential negative impact is that an adversary could preferentially target services that deploy aggressive relaxed verification.

- *End users* of LLM systems, whose safety depends on the target model’s alignment being preserved at inference time. They may benefit from defenses that keep the system’s behavior consistent with the safety-aligned target model, while unprotected lossy speculative decoding could expose them to more harmful responses or to unintended actions triggered by prompt injection in downstream applications.
- *Researchers and practitioners* studying LLM systems, who benefit from understanding that inference optimizations are security-relevant components. Our work characterizes the security–utility asymmetry and its origin in early-position draft tokens, and may motivate further defensive research, while this analysis could also guide attackers toward settings where existing defenses are weaker.

Mitigation. We took steps to minimize harm to the identified stakeholders and promote defensive progress.

- *For model providers and infrastructure operators, a practical defense:* Alongside the measurement, we propose SECURESD, which restores safety and security performance close to that of lossless speculative decoding while preserving efficiency and utility.
- *For end users, no real-world harm:* All experiments used public benchmarks (WildJailbreak, JailbreakBench, HarmBench, AdvBench, OpenPromptInjection, AgentDojo, HumanEval, and GSM8K) and open-weight models (Qwen3 and Llama3 families) deployed locally in controlled offline environments. We did not use private user data, interact with real users, or issue attack queries to production services. GPT-4o-mini is used only as an LLM-as-a-judge to classify model outputs, and we do not include harmful model generations in the paper.
- *For researchers and practitioners, no new attack technique:* All jailbreak and prompt injection inputs are drawn from existing public datasets, and our adaptive attack only applies a simple prefix-delaying transformation to them. We release the code of SECURESD and the JailbreakSD benchmark, which is constructed solely from existing public datasets, to support reproducible defensive research rather than to provide exploit guidance.

Recommended Future Safeguards. Real-world systems should incorporate safeguards addressing risks to both infrastructure operators and end users. Specifically, prior to deployment, systems utilizing relaxed verification must be evaluated against safety and security benchmarks alongside standard utility metrics, with acceptance rates capped at levels where any security degradation remains strictly tolerable.

Justification for Research. Lossy speculative decoding improves LLM inference efficiency and is already supported by mainstream serving frameworks, but its security implications remain largely unexplored. Identifying these risks and providing a practical defense is essential to ensure that efficiency gains do not silently compromise the safety and security of LLM systems.

Appendix D. Meta-Review

The following meta-review was prepared by the program committee for the 2027 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

D.1. Summary

The paper studies lossy speculative decoding and identifies a security–utility asymmetry. The resilience of the system against jailbreak and prompt-injection degrades substantially earlier and faster than standard utility metrics. This is attributed to the disproportionate impact (and susceptibility) of the early tokens. To address this, the work proposes a defence – SecureSD – the lowers attack effectiveness.

D.2. Scientific Contributions

- Identifies an Impactful Vulnerability
- Provides a Valuable Step Forward in an Established Field

D.3. Reasons for Acceptance

- 1) The work identifies a new threat by observing that under lossy speculative decoding, the susceptibility of the attacks degrades faster than model utility.
- 2) The work introduces the first defence to address this – SecureSD – that relies on interpolating between the draft and judge models to get the output distribution closer to that of lossless speculative decoding. It leverages the known insight that early tokens have higher impact on attack effectiveness.

D.4. Noteworthy Concerns

- 1) The security guarantees of SecureSD and the evaluation of the adaptive attacks are a good first step, but overall limited.

Appendix E. Response to the Meta-Review

We thank the reviewers and would like to clarify SECURESD’s scope. SECURESD mitigates the additional security degradation introduced by lossy speculative decoding, with target-only decoding as its security reference. It does not claim to eliminate the target model’s existing vulnerabilities or provide universal adversarial robustness.

Our adaptive evaluation directly targets SECURESD’s design by delaying malicious generation beyond its early-token correction window. The results in Table 12 show that SECURESD retains mitigation benefits under this defense-aware attack. We acknowledge that evaluating more diverse adaptive strategies is an important direction for future work.